

Full Stack Development with MERN

Project Title: BookStore MERN Application

College Name: Aditya College Of Engineering and Technology

Department: Department of Computer Science & Engineering

Course: Bachelor of Technology

Title: Full-Stack Web Development (MERN)

Project Guide:

Date of Submission:

Repository: <https://github.com/Tejaswini0425/BookStore-MERN.git>

1. Introduction:

Project Title: BookStore MERN Application

The BookStore MERN Application is a full-stack web application developed using the MERN technology stack:

- **MongoDB** – Database
- **Express.js** – Backend Framework
- **React.js** – Frontend Library
- **Node.js** – Runtime Environment

The application provides an online platform where users can explore books, view details, manage their accounts, and perform bookstore-related activities.

Team Members and Roles:

S.No	Team Member	Role	Responsibilities
1	Tejaswini Katta	Team Lead	Project planning, team coordination, backend integration, final project management
2	Boddani Sai Chanti	Full Stack Developer	Frontend development, backend API integration, database connectivity, testing and documentation
3	Motupalli Mahendra Jagadeesh	Frontend Developer	Designing React components, creating UI pages, improving user experience
4	Neyyala Naveen Kumar	Backend Developer	Developing REST APIs, server-side logic, authentication implementation
5	Bitra Pavan Santosh Kumar	Database & Testing Member	MongoDB database management, API testing, bug fixing and project support

2. Project Overview

Purpose:

The main purpose of the **BookStore MERN Application** is to develop an online bookstore system where users can easily search, view, and manage books digitally.

The project aims to replace traditional manual bookstore operations with an efficient web-based system.

The application provides:

- Easy access to books
- Digital book management
- User account management
- Secure authentication
- Efficient database handling

Features:

User Features:

- User registration
- User login/logout
- Browse available books
- Search books
- View book details
- Add books to cart
- Place orders
- Manage profile

Admin Features:

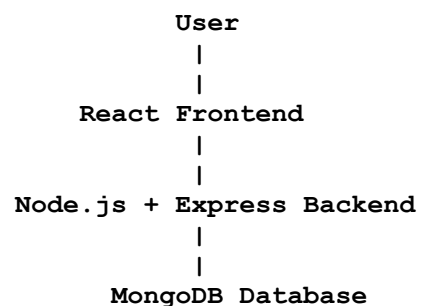
- Admin login
- Add new books
- Update book details
- Delete books
- Manage users
- Manage orders

Technical Features:

- Responsive React user interface
- REST API communication
- MongoDB database integration
- JWT-based authentication
- Secure password handling

3. Architecture

The project follows a three-layer architecture:



3.1 Frontend Architecture (React.js)

Technology Used:

- React.js
- JavaScript
- HTML
- CSS

- Axios
- React Router

Description:

The frontend is developed using React.js.

React components are used to create reusable and interactive user interface elements.

The frontend communicates with the backend using API requests.

Responsibilities:

- Display UI pages
- Handle user interactions
- Manage application navigation
- Send requests to backend APIs
- Display database information

Example structure:

client

|

|-- src

|

|-- components

|-- pages

|-- assets

|-- App.js

|-- index.js

3.2 Backend Architecture (Node.js & Express.js)

Technology Used:

- Node.js
- Express.js

Description:

The backend manages application logic and provides APIs for frontend communication.

It handles:

- User authentication
- Book management
- Order processing
- Database operations

Backend Flow:

Frontend Request

|

Express Router

|

Controller

|

MongoDB Database

|

Response

3.3 Database Architecture (MongoDB)

Database:

MongoDB is used as the database for storing application data.

MongoDB stores data in collections.

User Collection:

Stores user details.

Example:

```
{  
  "name": "Lalli",  
  "email": "lalli@gmail.com",  
  "password": "encrypted password"  
}
```

Book Collection:

Stores book information.

Example:

```
{  
  "title": "Java Programming",  
  "author": "James Gosling",  
  "price": 500,  
  "category": "Programming"  
}
```

Order Collection:

Stores order details.

Example:

```
{  
  "userId": "12345",  
  "bookId": "67890",  
}
```

```
"status": "Delivered"
```

```
}
```

4. Setup Instructions

Prerequisites:

Required software:

1. Node.js

Used for running frontend and backend servers.

2. MongoDB

Used for database storage.

3. Git

Used for downloading project source code.

4. Visual Studio Code

Used as development environment.

Installation Steps:

Step 1: Clone Repository

Open terminal:

```
git clone https://github.com/Tejaswini0425/BookStore-MERN.git
```

Step 2: Open Project Folder

```
cd BookStore-MERN
```

Step 3: Install Frontend Dependencies

Go to client folder:

```
cd client
```

Install packages:

```
npm install
```

Step 4: Install Backend Dependencies

Open another terminal:

```
cd server
```

Install packages:

```
npm install
```

Step 5: Environment Variables Setup

Create a .env file inside server folder.

Example:

```
PORT=5000
```

```
MONGO_URI=your_mongodb_connection_string
```

```
JWT_SECRET=your_secret_key
```

5. Folder Structure

Client Folder (React)

client

|

|-- public

|

|-- src

|

|-- components

|

|-- pages

|

|-- services

|

|-- App.js

|

|-- index.js

Components:

Contains reusable UI components.

Pages:

Contains application screens.

Example:

- Home Page
- Login Page
- Register Page
- Book Page
- Cart Page

Server Folder(Node.js)

```
server
|
|-- controllers
|
|-- models
|
|-- routes
|
|-- middleware
|
|-- server.js
|
|-- package.json
```

Controllers:

Handles business logic.

Models:

Defines MongoDB schemas.

Routes:

Defines API endpoints.

Middleware:

Handles authentication and security.

6. Running the Application

Start Backend:

Navigate to server folder:

```
cd server
```

Run:

```
npm start
```

Backend runs on:

```
http://localhost:5000
```

Start Frontend:

Navigate to client folder:

```
cd client
```

Run:

```
npm start
```

Frontend runs on:

```
http://localhost:3000
```

7. API Documentation

User Registration

Method:

```
POST
```

Endpoint:

```
/api/users/register
```

Request:

```
{  
  "name": "Sai",  
  "email": "sai@gmail.com",  
  "password": "123456"  
}
```

Response:

```
{  
  "message": "Registration successful"  
}
```

User Login

Method:

POST

Endpoint:

/api/users/login

Response:

```
{  
  "token": "JWT Token"  
}
```

Get Books

Method:

GET

Endpoint:

/api/books

Response:

```
[  
{  
  "title": "Python",  
  "price": 400  
}  
]
```

Add Book

Method:

POST

Endpoint:

/api/books/add

Delete Book

Method:

DELETE

Endpoint:

/api/books/:id

8. Authentication:

Authentication is implemented using **JWT (JSON Web Token)**.

Working Process:

1. User enters login credentials.
2. Server verifies user details.
3. JWT token is generated.

4. Token is stored on client side.
5. Token is sent with protected API requests.

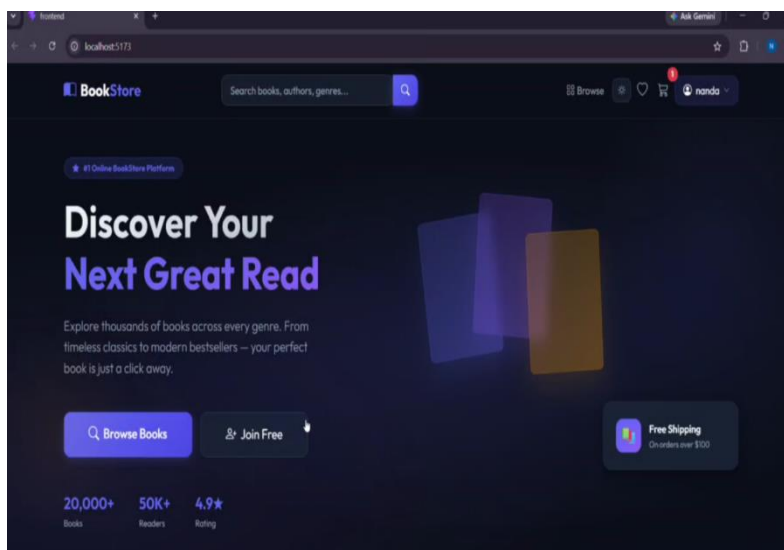
Security Features:

- Password encryption
- Protected routes
- Token verification
- User authorization

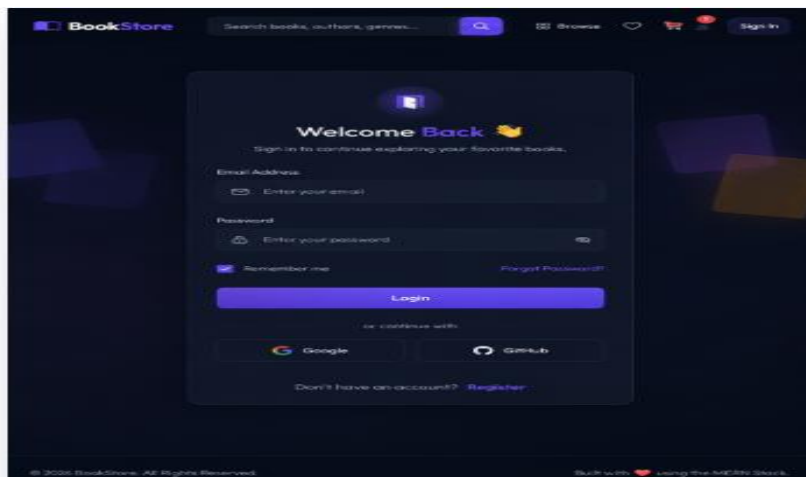
9.User Interface:

Screenshots should include:

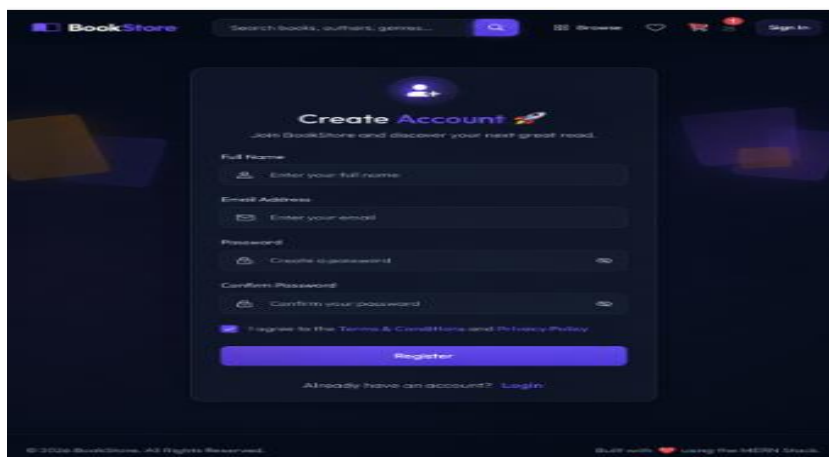
1.Home Page:



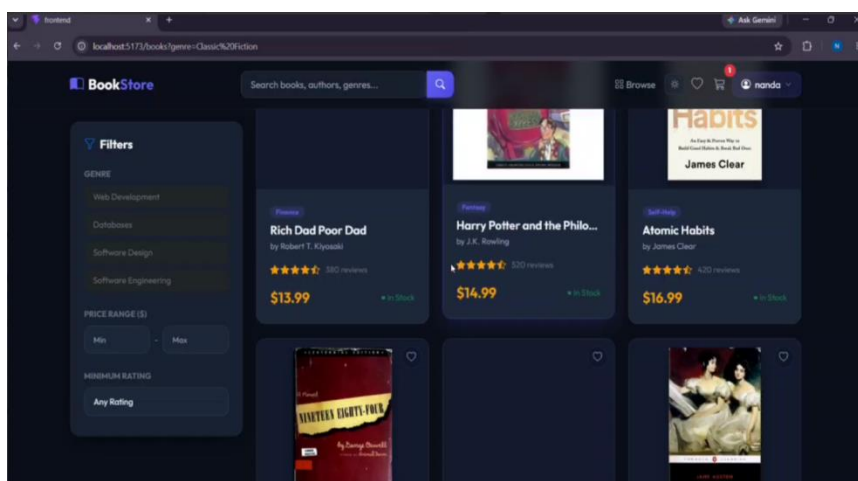
2.Login Page:



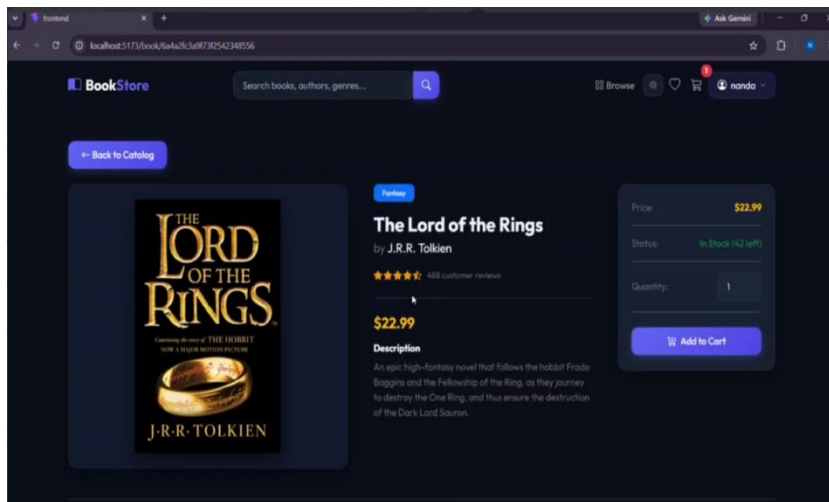
3.Registration Page:



4.Book Listing Page:



5.Cart Page Interface:



6.Admin Dashboard Interface:



10. Testing:

Testing Strategy:

The application is tested to ensure all features work correctly.

Testing Tools:

- Postman
- Browser Testing
- MongoDB Compass

Testing Types:

Functional Testing:

Testing:

- Login
- Registration
- Book operations
- Cart operations

API Testing:

All backend APIs are tested using Postman.

Demo

The project demonstration includes:

- Application screenshots
- User interface pages
- API testing results

Demo Link:

(<https://savedly.net/f/b87jr85t>)

12. Known Issues:

Current limitations:

- Online payment gateway is not integrated.
- Limited book categories.
- Email notifications are unavailable.
- Advanced recommendation system is not implemented.

13. Future Enhancements:

Future improvements:

- Add online payment system.
- Add book reviews and ratings.
- Add wishlist feature.

- Add AI-based book recommendations.
- Develop mobile application.
- Deploy application on cloud platforms.
- Improve security features.

Conclusion:

The **BookStore MERN Application** successfully demonstrates a complete full-stack web application using MongoDB, Express.js, React.js, and Node.js.

The project provides an efficient online bookstore platform with user management, book management, and secure authentication features.